Tabular Data Understanding

A comprehensive report submitted in partial fulfillment
of the requirements for the degree of

Doctor of Philosophy
in
Computer Science

by

Anupreksha Jain
2025801010
anupreksha.j@research.iiit.ac.in

Advisor: Dr. Manish Shrivastava



Language Technologies Research Center
International Institute of Information Technology Hyderabad
500 032, India

2026

Copyright © Anupreksha Jain, 2026
All Rights Reserved

Abstract

Tables are among the most pervasive formats for structured information, underpinning enterprise analytics, scientific reporting, and everyday data exchange. Yet reasoning over them with large language models (LLMs) is fundamentally difficult: models consume one-dimensional token sequences, whereas a table's meaning is carried jointly by its rows, columns, headers, and the relationships among cells. This report surveys how the field bridges that mismatch and organizes it around two complementary pillars: *schema understanding*, the mapping of natural-language intent onto a table's structural blueprint, and *data understanding*, the problem of reasoning faithfully over table content, whose canonical task is Table Question Answering.

We first review how two-dimensional tables are converted into representations that LLMs can consume (serialization, data schema, image, and specialized table encoders) and show that this choice conditions every downstream task. We then synthesize the literature under the two pillars, tracing a consistent shift from single-pass methods toward multi-step pipelines that decompose, retrieve, verify, and correct, and arguing that the binding constraints are increasingly architectural rather than a matter of raw model capacity.

Against this backdrop we present our contribution to the schema-understanding pillar, Reflect-SQL [1], a self-reflective Text-to-SQL system that constructs a semantic knowledge base over the schema, retrieves relevant elements iteratively, and synthesizes queries under syntactic and multi-dimensional semantic validation, scoring correctness with an LLM-as-a-judge and without a gold reference. By grounding reflection in the checkable signals the SQL setting affords, Reflect-SQL reaches 72.03% execution accuracy on the industry-scale BIRD benchmark.

Finally, we lay out an extensive research agenda across both pillars: for schema understanding, continually evolving knowledge bases, retrieval that scales by addressing pre-computed structure rather than searching at query time, cross-database join reasoning, and reliable yet inexpensive reflection; and for data understanding, our next direction, Table Question Answering as the target task and the evidence-retrieval bottleneck as a point of entry, approached by decoupling evidence selection from reasoning, with robustness on complex table structures and higher-order reasoning as the targets. The unifying thesis is that progress in tabular data understanding depends less on ever-larger models than on the architecture that supplies them with the right structure and the right evidence.

Contents

Chapter Page

1 Introduction . . . 1
1.1 Objective and scope . . . 2
1.2 Contributions . . . 3
1.3 Structure of the report . . . 3

2 Literature Survey . . . 5
2.1 Tabular Input Representations . . . 5
2.1.1 Serialization . . . 5
2.1.2 Data Schema . . . 6
2.1.3 Image . . . 6
2.1.4 Table Encoders . . . 6
2.1.5 Cross-cutting observations . . . 7
2.2 Schema Understanding . . . 7
2.2.1 Knowledge-base and metadata construction . . . 7
2.2.2 Schema linking and retrieval . . . 8
2.2.3 From encoder–decoder models to LLM pipelines . . . 8
2.2.4 Self-reflection for structured generation . . . 8
2.3 Data Understanding . . . 9
2.3.1 Table Question Answering . . . 9
2.3.2 Adjacent tasks . . . 9
2.3.3 Open challenges . . . 10
2.4 Graph Representations . . . 10

3 Current Progress: Reflect-SQL . . . 12
3.1 Framework Overview . . . 12
3.2 Knowledge Base Construction and Refinement . . . 13
3.2.1 Construction . . . 13
3.2.2 Iterative refinement . . . 13
3.3 Iterative RAG Retrieval . . . 14
3.3.1 Hierarchical table and column retrieval . . . 14
3.3.2 Relevance and completeness scoring . . . 14
3.3.3 Query refinement . . . 15
3.4 SQL Query Generation . . . 15
3.4.1 Syntactic validation and correction . . . 15
3.4.2 Semantic validation . . . 16

3.5 Entailment . . . 16
3.5.1 Entailment scoring . . . 17
3.5.2 Entailment decision . . . 17
3.6 Mitigating Bias in LLM-based Evaluation . . . 17
3.7 Model Parameter Optimization . . . 17
3.8 Evaluation . . . 18
3.8.1 Experimental setup . . . 18
3.8.2 Results . . . 18
3.8.3 Human evaluation . . . 18
3.8.4 Comparison with state of the art . . . 19
3.9 Ablation Study . . . 19
3.10 Summary . . . 20

4 Future Directions . . . 21
4.1 Future Directions in Schema Understanding . . . 21
4.1.1 From static to continually evolving knowledge bases . . . 21
4.1.2 Retrieval at industry scale . . . 22
4.1.3 Cross-database and multi-table join reasoning . . . 22
4.1.4 Reliable, reference-free verification . . . 22
4.1.5 The cost of reflection . . . 23
4.1.6 Beyond SQL . . . 23
4.2 Setting a Direction for Data Understanding . . . 23
4.2.1 Evidence retrieval as the central bottleneck . . . 23
4.2.2 Robustness on complex table structures . . . 24
4.2.3 Reasoning beyond retrieval and arithmetic . . . 24
4.2.4 Generalization across representations . . . 25
4.3 Summary . . . 25

5 Conclusion . . . 26

Bibliography . . . 28

List of Figures

Figure Page

3.1 High-level architecture of the Reflect-SQL framework. 13
3.2 Example JSON specification of the knowledge base. 14
3.3 Iterative retrieval loop. 15
3.4 Iterative SQL generation loop. 16

List of Tables

Table Page

3.1 Example of knowledge base refinement for BIRD [8]. 14
3.2 Optimized parameters using grid search. 17
3.3 Accuracy (%) of various models on the BIRD dataset. Top performers in each category are highlighted. 18
3.4 Example of SQL refinement driven by entailment feedback. 19
3.5 Comparison with state-of-the-art methods on the BIRD dev set (execution accuracy %). 19
3.6 Ablation results showing the contribution of each module. 20

List of Selected Papers

The following are the key papers that this report draws upon and that form the basis for discussion, organized by the two pillars around which the report is structured.

Foundations and Representations

[S1] X. Wu, A. Ritter, and W. Xu, **"Tabular Data Understanding with LLMs: A Survey of Recent Advances and Challenges"**, *arXiv:2508.00217*, 2025.

[S2] Y. Sui, M. Zhou, M. Zhou, S. Han, and D. Zhang, **"Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study"**, in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*, 2024.

Schema Understanding (Text-to-SQL)

[S3] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, et al., **"Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task"**, in *Proceedings of EMNLP*, 2018.

[S4] J. Li, B. Hui, G. Qu, J. Yang, B. Li, et al., **"Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD)"**, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[S5] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, et al., **"CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL"**, *arXiv:2410.01943*, 2024.

[S6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, **"Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection"**, *arXiv:2310.11511*, 2023.

Data Understanding

[S7] P. Pasupat and P. Liang, **"Compositional Semantic Parsing on Semi-Structured Tables"**, in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2015.

[S8] Z. Cheng, H. Dong, Z. Wang, R. Jia, J. Guo, et al., **"HiTab: A Hierarchical Table Dataset for Question Answering and Natural Language Generation"**, in *Proceedings of ACL*, 2022.

[S9] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, et al., **"TabFact: A Large-Scale Dataset for Table-Based Fact Verification"**, in *International Conference on Learning Representations (ICLR)*, 2020.

Graph Representation of Tabular Data

[S10] Q. Li, C. Huang, S. Li, Y. Xiang, D. Xiong, and W. Lei, **"GraphOTTER: Evolving LLM-Based Graph Reasoning for Complex Table Question Answering"**, in *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.

[S11] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, **"G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering"**, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[S12] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, et al., **"From Local to Global: A Graph RAG Approach to Query-Focused Summarization"**, *arXiv:2404.16130*, 2024.

Chapter 1

Introduction

Tables are among the most pervasive means of organizing information. From relational databases that power enterprise systems, to spreadsheets used in finance and science, to the info-boxes and result tables embedded in web pages and research papers, a large fraction of the world's structured knowledge is stored in tabular form. Enabling machines to *understand* this data (to answer questions over it, verify claims against it, summarize it, and translate natural-language intent into precise queries against it) is therefore a foundational problem for natural language processing and a prerequisite for democratizing access to data.

The recent progress of Large Language Models (LLMs) and Multimodal Large Language Models (MLLMs) has renewed interest in tabular data, but tables expose a fundamental mismatch with how these models process input. LLMs are trained on *linear*, one-dimensional token sequences, whereas a table is inherently *two-dimensional*: its meaning is carried jointly by rows, columns, headers, and the relationships between cells [2]. Compounding this, tables are structurally *flexible*. They range from well-normalized relational database tables, to hierarchical and multi-layered spreadsheets, to semi-structured info-boxes linked to free text. This diversity of format and purpose has driven the field toward a fragmented landscape of specialized representations, tasks, and datasets rather than universal solutions, making the area difficult to navigate and models difficult to generalize [2].

Before any reasoning can occur, a two-dimensional table must be converted into a form a model can consume. The choice of this input representation is not neutral: recent studies show that model performance can swing substantially depending on how closely the chosen format aligns with the data seen during pre-training [3]. The literature broadly distinguishes four families of representation: *serialization* (e.g., HTML, JSON, Markdown, or LaTeX renderings of the full table), *data schema* (the structural blueprint of the data, such as SQL `CREATE` statements or pandas dataframes), *image*-based formats, and specialized *table encoders* [2]. These representations trade off completeness against compactness: serialization and images preserve content but are bounded by context length and resolution, whereas a schema-based view bypasses length limits by exposing only structure, at the cost of the concrete values.

This report studies tabular data understanding through two complementary lenses that together span this space:

- **Schema understanding** is concerned with the *structural blueprint* of tabular data (table and column names, data types, primary and foreign keys, and inter-table relationships) and with mapping a user's natural-language intent onto that structure. Real databases are large and their naming is often cryptic and undocumented (e.g., columns named `A2` or `dname`), so the central challenges are disambiguating obscure schemas, retrieving the relevant schema elements for a query, and generating structurally and semantically valid operations such as SQL over them. Text-to-SQL is the canonical task in this pillar.

- **Data understanding** is concerned with reasoning over the *content* of tables (the values, their aggregation, and the trends, outliers, and relationships they encode) often in combination with surrounding text. Here the challenges are higher-order reasoning that goes beyond retrieval and arithmetic, and robustness to complex inputs such as hierarchical, long, and multi-table settings. Table Question Answering is the canonical task in this pillar, with table fact verification and table-to-text generation as adjacent tasks.

These two pillars are complementary rather than disjoint. A schema is a lossy abstraction of the underlying data, while the data is only interpretable in light of its schema; robust tabular understanding requires both. The current state of the field, however, is uneven. On the schema-understanding side, Text-to-SQL has advanced rapidly from early encoder–decoder architectures [4] to LLM-based in-context and multi-agent pipelines [5–7], with strong results on benchmarks such as BIRD [8]. On the data-understanding side, several benchmarks are approaching saturation on retrieval-oriented questions, yet models remain markedly weaker than humans once inputs become structurally complex: on hierarchical and multi-table benchmarks such as HiTab [9] and MULTIHIERTT [10], reported accuracies fall well below human performance. This gap motivates treating data understanding as the natural next frontier once schema understanding is well founded.

1.1 Objective and scope

The objective of this report is to survey and consolidate current research on tabular data understanding with large language models, organized around the two pillars of schema understanding and data understanding, and to situate our own research within this landscape. The report is scoped as follows:

- We adopt the taxonomy of tabular input representations (serialization, data schema, image, and table encoders) as the organizing framework for how tables are presented to models, and examine the sensitivity of model behaviour to these choices.

- Under **schema understanding**, we review knowledge-base and metadata construction for obscure schemas, schema linking and retrieval, and the evolution of Text-to-SQL methods, culminating in retrieval-augmented and self-reflective LLM pipelines.

- Under **data understanding**, we center on Table Question Answering, noting table fact verification and table-to-text as adjacent tasks, and review the open challenges of reasoning beyond retrieval, robustness to complex inputs, and generalization across tabular representations.
- As current progress, we present *Reflect-SQL*, a self-reflection-based framework for Text-to-SQL that operationalizes the schema-understanding pillar, and we use it to motivate data understanding as the forward research direction.
- The report focuses on methods, tasks, and benchmarks at the level needed to frame a research programme; it is not intended as an exhaustive technical account of every approach.

1.2 Contributions

The following are the contributions of this report:

- A structured survey of tabular data understanding that unifies the fragmented literature under two complementary pillars, schema understanding and data understanding, grounded in a common taxonomy of tabular input representations.
- A synthesis of the Text-to-SQL and schema-understanding literature, tracing the progression from encoder–decoder models to knowledge-base-driven, retrieval-augmented, and self-reflective LLM frameworks, and identifying where reliability gaps remain.
- A synthesis of the data-understanding literature that consolidates the principal reasoning tasks and articulates three open challenges: limited reasoning beyond retrieval, lack of robustness on complex inputs, and limited cross-representation generalization.
- *Reflect-SQL* [1], our framework for enterprise Text-to-SQL, which introduces a dynamically enriched knowledge base, a feedback-driven retrieval loop, an iterative SQL synthesis loop, and a reference-free entailment check, and which achieves an execution accuracy of 72.03% on the BIRD benchmark.
- An extensive research agenda spanning both pillars: for schema understanding, continually evolving knowledge bases, retrieval that scales by addressing pre-computed structure rather than searching at query time, cross-database join reasoning, and reliable yet inexpensive reflection; and for data understanding, the identification of evidence retrieval as the central bottleneck and a concrete direction (decoupling evidence selection from reasoning) for attacking it.

1.3 Structure of the report

The remainder of this report is organized into four chapters. Chapter 2 presents the literature survey: it first introduces the taxonomy of tabular input representations, then reviews the schema-understanding

pillar (knowledge-base construction, schema linking and retrieval, and the evolution of Text-to-SQL from encoder–decoder models to self-reflective LLM pipelines) and the data-understanding pillar, centered on Table Question Answering, and its open challenges) establishing the research context for both pillars in a balanced manner. Chapter 3 reports current progress: it details *Reflect-SQL*, our self-reflection framework for Text-to-SQL, including its knowledge base, retrieval and generation loops, reference-free entailment check, and experimental results on the BIRD benchmark. Chapter 4 sets out an extensive research agenda: it consolidates what schema understanding has achieved and the problems that remain, then charts our next direction in data understanding, identifying the evidence-retrieval bottleneck as the point of entry. Chapter 5 concludes by summarizing the findings and restating the report's central thesis: that progress in tabular data understanding depends less on ever-larger models than on the architecture that supplies them with the right structure and the right evidence.

Chapter 2

Literature Survey

This chapter surveys the research landscape of tabular data understanding. We first examine how two-dimensional tables are converted into representations that large language models can consume (Section 2.1), since this choice conditions every downstream task. We then review the two pillars that structure this report: *schema understanding*, which concerns the structural blueprint of tabular data and the mapping of natural-language intent onto it (Section 2.2); and *data understanding*, which concerns reasoning over the content of tables, canonically instantiated as Table Question Answering, and the open challenges that remain (Section 2.3). Rather than reviewing each paper in isolation, we synthesize the literature thematically, grouping works by the problem they address and drawing out the trends that connect them.

2.1 Tabular Input Representations

Large language models operate on one-dimensional token sequences, whereas tables are inherently two-dimensional: their meaning is carried jointly by rows, columns, headers, and the relationships among cells. Any tabular understanding pipeline must therefore first transform the table into a linear input, a step that frequently discards structural information [2]. This transformation is consequential rather than cosmetic: model performance is sensitive to the chosen representation, varying by several percentage points depending on how closely the input format aligns with the data encountered during pre-training [3]. The literature distinguishes four broad families of representation (serialization, data schema, image, and specialized table encoders), which we review in turn.

2.1.1 Serialization

Serialization renders a table as text using an established markup or structured format, such as HTML, Markdown, JSON, LaTeX, or comma- and row-separated variants [2]. Its principal advantage is compatibility: such formats occur naturally on the web, in documents, and in code environments, so models encounter them during pre-training. Most current benchmarks therefore adopt serialization as the default representation. Different formats capture structure to differing degrees; for instance, LaTeX's `multicolumn` construct expresses hierarchical headers that flatter formats lose during serialization.

Beyond the choice of format, the finer details of *input design* (the ordering of rows and columns, the marking of partitions, and the inclusion of illustrative example rows) also cause substantial performance fluctuations across table-interaction tasks such as cell lookup and reverse lookup [3].

A practical limitation of serialization is length. Long or multi-table inputs can exceed a model's context window, causing truncation and information loss. To cope, researchers sample the rows or columns most relevant to a query; embedding-based strategies such as centroid and semantic sampling have been shown to outperform naive selection, and combining sampled content with lightweight augmentations (table size, keyword explanations) improves performance within a fixed token budget [2].

2.1.2 Data Schema

An alternative to presenting the full table content is to present only its *schema*: the structural blueprint of the data, expressed for example as SQL `CREATE TABLE` statements or as pandas dataframe declarations [2]. By exposing table names, column names, data types, and key constraints while omitting the bulk of the values, a schema-based representation sidesteps context-length limits and focuses the model on structure. This makes it the natural representation for the schema-understanding pillar, and it underpins the strong results obtained when models interact with tables through generated SQL or Python code. The trade-off is a loss of concrete content: schema-only views discard the values that are often essential for disambiguation, and they presuppose well-structured tables to be effective. The design of the schema view matters as well; supplying a few example rows alongside the schema improves results, and performance degrades sharply when primary- and foreign-key information is omitted, since key relationships are what make join reasoning possible [3].

2.1.3 Image

With the advent of multimodal LLMs, rendering a table as an image has emerged as a third representation. Images preserve the original visual layout (including merged cells and hierarchical headers) that serialization may distort, and they are readily obtained from documents. Fine-tuning multimodal models with auxiliary objectives focused on table structure, such as cell extraction and cell-location prediction, has yielded results that surpass OCR-plus-serialization pipelines [2]. The limitation mirrors that of serialization: the information an image can convey is bounded by its resolution, so as tables grow the content blurs and performance deteriorates. A promising but under-explored direction is to combine an image (for global structure) with a text-based view (for precise content) as complementary inputs.

2.1.4 Table Encoders

A final family embeds structure directly into the model through specialized encoders. Smaller-scale models have used column-based, row-based, tree-structured, and graph-based embeddings to capture the two-dimensional layout of tables. More recently this idea has scaled: table *foundation models* integrate

a specialized encoder with a large base model, incorporating tabular data during pre-training and fine-tuning. TableGPT2, for example, couples a table encoder that uses column- and row-wise attention with 7B- and 72B-parameter base models, outperforming generalist table models across a range of tasks while remaining competitive with task-specific methods [11]. This trend signals a move away from purely prompt-based table processing toward architectures that treat tabular structure as a first-class modality.

2.1.5 Cross-cutting observations

Two observations recur across all four families. First, current benchmarks tend to select representations for convenience rather than through principled, unbiased comparison, so the same task appears with incompatible formats across datasets, complicating fair evaluation [2]. Second, no single representation dominates: each trades completeness against compactness and structural fidelity differently, which motivates the serialization-to-serialization and multi-representation training directions discussed later in this chapter. These observations set the stage for the two pillars, which differ precisely in which representation they privilege: schema for the former, content for the latter.

2.2 Schema Understanding

Schema understanding concerns the structural blueprint of tabular data (table and column names, data types, primary and foreign keys, and inter-table relationships) and the task of mapping a user's natural-language intent onto that structure. Its canonical instantiation is Text-to-SQL: translating a natural-language question into an executable query over a relational database. Deploying this capability in realistic settings is hard for three reasons: schemas are large and their naming is often cryptic and undocumented; the relevant schema elements must be retrieved before a query can be formed; and the generated query must be not merely syntactically valid but logically faithful to the user's intent [8]. We organize the literature around how it addresses these three challenges.

2.2.1 Knowledge-base and metadata construction

The first challenge (obscure schemas) has motivated a line of work that enriches the raw schema with a semantic layer before any query is attempted. Such knowledge bases are built by aligning schema metadata with business glossaries [12], by automatically generating column and table descriptions [13], or by injecting domain knowledge from external documents [14]. These descriptions disambiguate cryptic identifiers (for example, resolving a column named `A2` to its business meaning) and externalize schema semantics into a structured artifact that a model can consult, which is essential when industry-scale schemas exceed the model's context window. A recurring limitation of this line of work is that it treats the knowledge base as a *static* asset, constructed once and thereafter fixed; it does not adapt as new queries reveal previously undocumented aspects of the schema.

2.2.2 Schema linking and retrieval

The second challenge (identifying which tables and columns are relevant to a given question) is a retrieval problem. Retrieval-Augmented Generation (RAG) [15] provides the general template of grounding generation in retrieved context, and dense retrievers built on sentence embeddings [16] make it possible to match a natural-language query against schema-element descriptions by semantic similarity. For databases, retrieval must respect structure: relevant tables imply related tables through foreign-key relationships, so join-aware and hierarchical retrieval (first selecting tables, then narrowing to columns, and expanding along relationships to preserve join paths) is more effective than flat retrieval. A central difficulty is that vague user queries map poorly onto structured schemas, which has prompted iterative approaches that reformulate the query when the retrieved context is judged insufficient, echoing self-reflective RAG methods developed for open-domain settings [17].

2.2.3 From encoder–decoder models to LLM pipelines

The third challenge (generating faithful queries) has driven the methodological evolution of the field. Early neural systems framed Text-to-SQL as sequence generation: Seq2SQL used reinforcement learning to generate structured queries [4], and SQLNet replaced generation with a sketch-based slot-filling formulation to avoid the pitfalls of ordering in the reward [18]. The introduction of large, cross-domain benchmarks such as Spider [19] and, more recently, the industry-scale BIRD benchmark [8] shifted evaluation toward generalization across unseen schemas and toward execution accuracy on realistic databases. With LLMs, the dominant paradigm became in-context learning: DIN-SQL decomposes the task into sub-problems and adds self-correction [5]; DAIL-SQL systematically studies prompt construction and exemplar selection [20]; SQLPrompt explores few-shot prompting for the task [21]; MAC-SQL casts generation as a collaboration among specialized agents (selector, decomposer, refiner) [6]; and CHASE-SQL generates multiple candidate queries via diverse reasoning paths and selects among them with a preference-optimized selector [7]. The clear trend is from single-pass generation toward multi-step pipelines that decompose, verify, and correct.

2.2.4 Self-reflection for structured generation

A unifying thread across the most recent methods is *self-reflection*: rather than trusting a single generation, the model scores its own output and iteratively refines it. The idea originates in open-domain generation, where self-reflection improves factuality and consistency by having a model critique and revise its answers [22], and where self-reflective retrieval decides when and what to retrieve [17]. Adapting this paradigm to Text-to-SQL is non-trivial because the domain offers concrete, checkable signals (a query can be executed, its syntax validated, and its results inspected) that open-domain generation lacks. Using an LLM as an evaluator (LLM-as-a-judge) makes it possible to score retrieval relevance, semantic correctness, and result–intent alignment, though such judges must be prompted carefully with rubrics and examples to mitigate the biases inherent in LLM-based evaluation [23]. Our own work, described

in the next chapter, builds directly on this thread by integrating reflection at the retrieval, generation, and verification stages of a Text-to-SQL pipeline.

2.3 Data Understanding

Data understanding concerns reasoning over the *content* of tables (their values and the aggregations, comparisons, trends, and outliers they encode), frequently in combination with surrounding text. Whereas schema understanding privileges the structural blueprint, data understanding privileges the cells themselves and the semantics that connect them. Its canonical instantiation, and the task we concentrate on, is Table Question Answering; we treat it as the representative task for the pillar, note two adjacent tasks (fact verification and table-to-text) for context, and then consolidate the open challenges that define our forward direction.

2.3.1 Table Question Answering

Table Question Answering (TQA) is the most studied task: given a natural-language question and one or more tables, sometimes accompanied by a passage, the model produces an answer derived from the table content, typically as cell spans, computed values, or short text spans [2]. The benchmark landscape has expanded along several axes. Foundational datasets such as WikiTableQuestions [24] and the sequential SQA [25] established open-domain answering over web tables. Subsequent benchmarks pushed on *domain* (finance in TAT-QA [26] and FinQA [27]), on *input complexity* (hybrid tabular–textual reasoning in HybridQA [28], hierarchical tables in HiTab [9], and multi-hierarchical and multi-table settings in MULTIHIERTT [10] and MultiTabQA [29]), and on *answer format* (free-form generation in FeTaQA [30]). A notable methodological finding is that decomposition-based pipelines, which break a question into simpler steps executed against the table, have driven large gains and now approach saturation on the simpler retrieval-oriented benchmarks.

2.3.2 Adjacent tasks

Two further tasks round out the data-understanding pillar but are not our focus. *Table Fact Verification* (TFV), also called table-based natural language inference, asks whether a claim is supported, refuted, or unverifiable given a reference table; TabFact established the task at scale over Wikipedia tables [31], and modern methods exceed 80% accuracy, though the strong numbers concentrate on well-structured single tables and thus test fact retrieval more than deep reasoning. *Table-to-Text* generation instead asks a model to describe table content faithfully in natural language, ranging from controlled descriptions of highlighted cells (ToTTo [32]) to query-focused summaries that must reason about underlying causes rather than surface content (QTSUMM [33]). Both tasks share Table Question Answering's core difficulty, isolating and reasoning over the relevant cells, which is why we take Table QA as the representative task for the pillar.

2.3.3 Open challenges

Synthesizing across tasks, the survey literature identifies three challenges that define the frontier of data understanding [2].

First, limited scope beyond retrieval and arithmetic. Many benchmarks are constructed by translating SQL queries or mathematical expressions into natural-language questions, so their core structure rewards retrieval and computation rather than higher-order reasoning. As several such benchmarks approach saturation, recent work has begun to demand diagnostic, predictive, and prescriptive thinking (identifying insights, forecasting, and generating charts from ambiguous requests) and intent-driven querying, where a system must infer what an abstract, goal-oriented request actually asks for [34, 35]. These tasks cannot be solved by matching synonyms to table headers; they require understanding the table's overall context and the user's intent.

Second, lack of robustness on complex inputs. Intricate table structures, long tables, lengthy accompanying texts, and multi-table settings degrade model performance far more than they degrade human performance. On hierarchical and multi-table benchmarks such as HiTab [9] and MULTI-HIERTT [10], reported accuracies fall below 50% against human performance around 83%, and even on well-structured multi-table QA the strongest evaluated models reach only about half of human accuracy. Robustness to input complexity is thus a first-order open problem.

Third, limited generalization across representations. As noted in Section 2.1, performance varies by up to several percentage points with the input format, and models struggle to transfer across the diverse representations used by different benchmarks [3]. Proposed remedies include standardizing serialization options, deriving serialized views from images of complex tables, and training on serialization-to-serialization tasks that convert one format into another to build robustness to varied input styles.

Together these challenges frame data understanding as the natural progression from schema understanding: once a system can reliably map intent onto structure, the harder problem is reasoning faithfully over content at the scale and complexity of real-world tables.

2.4 Graph Representations

A representation that cuts across both pillars, and that has attracted growing attention, casts the table itself as a *graph*: cells, rows, headers, and their groupings become typed nodes, and the relational topology of the table (which cell lies in which row and column, and how headers span sub-columns) is encoded as edges. Unlike the linear formats of Section 2.1, a graph makes a table's structure explicit and addressable rather than implicit in a flattened token stream, and it composes naturally with graph-based retrieval, which makes it especially relevant to Table Question Answering over large and hierarchical tables, where the answer-bearing cells must be located before the model can reason over them.

Graph Retrieval-Augmented Generation replaces full-table linearization by first filtering a relevant subgraph before the model consumes it: GraphRAG builds a graph index to support query-focused retrieval [36], and G-Retriever retrieves relevant subgraphs for question answering over textual graphs [37].

Agentic extensions push further by treating the model as a graph walker: GraphOTTER, for instance, performs LLM-guided reasoning over a graph representation of a complex table, navigating from headers to the cells that answer a query [38]. The appeal of these approaches is structure-aware evidence selection (delivering only the relevant region of a large table to the model), which speaks directly to the robustness and reasoning challenges identified above. Their principal cost is efficiency: scoring each traversal step with the model ties latency to table size, a tension we return to as a central open problem for data understanding in Chapter 4.

Having surveyed how tables are represented, how the two pillars structure the field, and where its limits lie, the next chapter turns to our own contribution to the schema-understanding pillar, which lays the groundwork for pursuing this progression.

Chapter 3

Current Progress: Reflect-SQL

This chapter presents our research contribution to date, *Reflect-SQL* [1], a framework for Text-to-SQL that operationalizes the schema-understanding pillar developed in Chapter 2. As established there, deploying Text-to-SQL in realistic settings is hindered by three systemic difficulties: (i) **schema obscurity**, the large-scale databases with cryptic column names (e.g., `A2`, `dname`) and undocumented relationships; (ii) **contextual retrieval**, the difficulty of extracting the relevant schema elements when vague user queries map poorly onto structured schemas; and (iii) **logical integrity**, the need to move beyond syntactic validity to guarantee that a query faithfully answers the user's intent. Reflect-SQL addresses these through a multi-stage *self-reflection* approach that, rather than trusting a single-pass generation, scores its own intermediate outputs and iteratively refines them within interconnected feedback loops. This directly extends the self-reflection thread of Section 2.2 to the structured domain of databases, exploiting the concrete, checkable signals (executability, syntax, and result inspection) that this domain uniquely affords.

The primary contributions of the framework are: (1) a complete, end-to-end Text-to-SQL system for enterprise settings; (2) a **dynamic knowledge base** that enriches schema understanding over successful executions; (3) a **reference-free** methodology for validating SQL without gold-standard labels; and (4) a **feedback-driven query refinement** process that aligns ambiguous user queries with the database schema. On the challenging BIRD benchmark [8], Reflect-SQL attains an execution accuracy of 72.03%, substantially outperforming strong baselines.

3.1 Framework Overview

The framework is organized as a knowledge base feeding three interconnected reflective loops (retrieval, SQL generation, and entailment) as depicted in Fig. 3.1. A dynamic **knowledge base** (KB) acts as a semantic abstraction layer over the raw schema. A **feedback-driven retrieval** loop then identifies the tables and columns relevant to a query, an **iterative SQL synthesis** loop generates and corrects a candidate query, and a final **entailment** check verifies that the executed result matches the user's intent before returning it and enriching the KB.

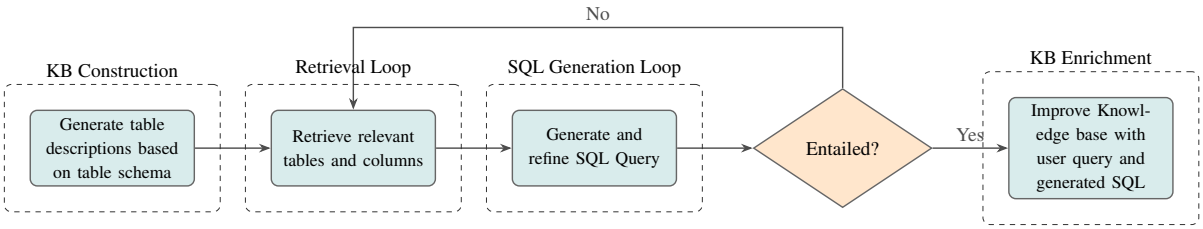


Figure 3.1 High-level architecture of the Reflect-SQL framework.

3.2 Knowledge Base Construction and Refinement

The knowledge base is a semantic abstraction layer containing table descriptions, column metadata, and relationships. It is vital for industry-scale databases, which routinely exceed the context limits of LLMs and suffer from obscure naming conventions. By externalizing schema details into a structured KB, the framework overcomes context constraints and limited schema understanding, ensuring the LLM accesses only the specific information needed to bridge user intent and the database. This design responds directly to the static-KB limitation identified in Section 2.2: the KB here is not a one-time artifact but is continuously refined.

3.2.1 Construction

For the BIRD dataset [8], the KB is constructed by synthesizing the database schema, sample data values, and the provided training questions and external evidence. In broader enterprise settings, the same process ingests domain knowledge from business documents and glossaries. These inputs are processed to generate comprehensive descriptions for tables, columns, and their relationships. To enable efficient retrieval, the components are encoded into dense vector representations using a sentence embedding model [16]. The vector database is structured hierarchically: a global collection for table descriptions and separate local collections for each table's column descriptions. This supports a two-step retrieval process: first identifying relevant tables, then narrowing to specific columns. A representative JSON structure of the KB is shown in Fig. 3.2.

3.2.2 Iterative refinement

In a post-processing step, after a successful SQL execution, one that passes syntactic and semantic validation and whose entailment score exceeds a threshold θ_e, the knowledge base is refined. This conditional update is critical for preventing error propagation: low-quality or contextually incorrect queries could otherwise pollute the KB, so only validated queries with validated results contribute to the system's learning. This mechanism is essential in real-world environments with ambiguous column names and continuously evolving schemas. Table 3.1 illustrates how column descriptions are enriched after a successful query.

```
{
  "financial": {
    "account": {
      "table_description": "it stores information about bank accounts ...",
      "relationships": [{
          "column_name": "district_id", "related_table": "district",
          "related_column": "district_id" }...],
      "column_description": {
        "account_id": "Unique identifier for each bank account...",
        "district_id": "Identifier representing the district...",
        ...}
    }...}
  ...
}
```

Figure 3.2 Example JSON specification of the knowledge base.

Component	Before Refinement	After Refinement
`loan.status`	Categorical column for loan status with values 'A', 'B', 'C', 'D'.	Loan repayment state: 'A' indicates finished or paid, 'B' indicates defaulted...
`trans.k_symbol`	Text column named 'k_symbol' with values 'POJISTNE', 'SIPO'.	Transaction Category: determines the payment type (e.g. 'POJISTNE' maps to Insurance)...

Table 3.1 Example of knowledge base refinement for BIRD [8].

3.3 Iterative RAG Retrieval

In the retrieval phase, the curated KB is used to identify the tables and columns relevant to a user query, as detailed in Fig. 3.3.

3.3.1 Hierarchical table and column retrieval

Given a user query Q, we embed it and use cosine similarity (Eq. 3.1) to retrieve the top-k tables $\{T_1, \ldots, T_k\}$ from the vector database.

$$\text{score}(T_i, Q) = \cos(\vec{t_i}, \vec{q}) = \frac{\vec{t_i} \cdot \vec{q}}{\|\vec{t_i}\| \|\vec{q}\|} \tag{3.1}$$

This set is expanded by including tables related via relationships defined in the KB, ensuring join paths are available, the join-aware retrieval principle discussed in Section 2.2. For each selected table, a similar embedding-based search retrieves the most relevant columns.

3.3.2 Relevance and completeness scoring

We assess retrieval quality with two metrics: a **relevance score** (R) for semantic alignment and a **completeness score** (C) to ensure all necessary schema elements are present. These are produced by an

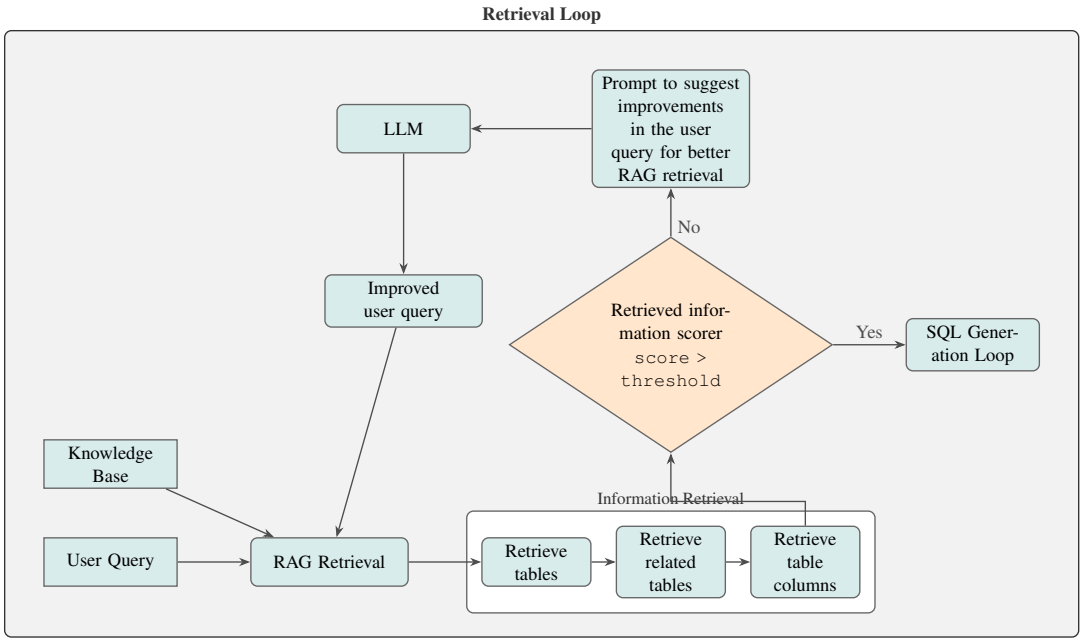


Figure 3.3 Iterative retrieval loop.

LLM-as-a-judge, and the final retrieval score $Score_r$ is a convex combination (Eq. 3.2).

$$Score_r = \alpha R + (1 - \alpha)C, \quad \text{where} \quad \alpha \in [0, 1] \tag{3.2}$$

3.3.3 Query refinement

If the retrieved schema is insufficient ($Score_r < \theta_r$), an iterative refinement loop is triggered. In each iteration, the LLM reformulates the user query into a more detailed version Q' to sharpen its semantic focus, while the retrieval scope is simultaneously widened by increasing the number of tables and columns retrieved in the next pass. Refining the query's focus while broadening the search area lets the system converge on the complete set of schema elements required for complex queries. The loop terminates once $Score_r \geq \theta_r$ or a maximum number of iterations is reached.

3.4 SQL Query Generation

With a high-quality set of retrieved tables, columns, descriptions, and sample data, a candidate SQL query is generated by an LLM and then subjected to two nested validation loops, as shown in Fig. 3.4.

3.4.1 Syntactic validation and correction

Each generated query is immediately checked for syntactic validity using a database engine. On a syntax error, a correction loop feeds the original query, the error message, and the schema context back

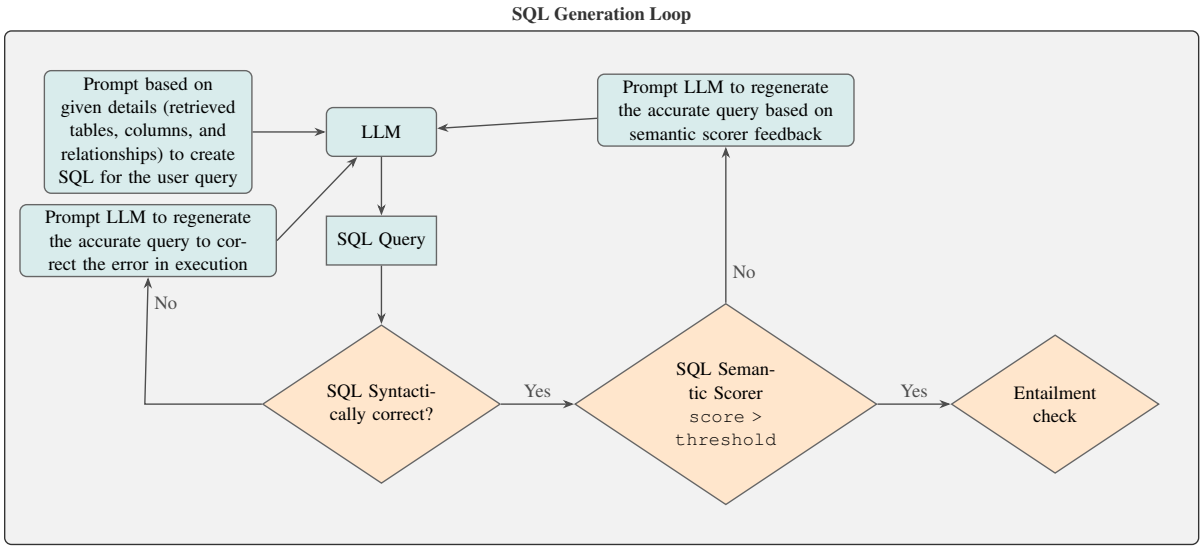


Figure 3.4 Iterative SQL generation loop.

to the LLM with instructions to fix the error. The loop repeats until a syntactically correct query is produced or a maximum iteration limit is met, making the system resilient to basic generation mistakes.

3.4.2 Semantic validation

A syntactically valid query undergoes semantic validation across four dimensions: column-name consistency (C), granularity and aggregation (G), join and relationship consistency (J), and query-logic alignment (Q). An LLM-as-a-judge scores each dimension, and a final semantic score S is a weighted sum (Eq. 3.3).

$$S = w_c \cdot C + w_g \cdot G + w_j \cdot J + w_q \cdot Q \tag{3.3}$$

where w_c, w_g, w_j, w_q are predefined weights. If $S < \theta_s$, the query is returned to the LLM with diagnostic feedback for iterative correction.

3.5 Entailment

Once a query passes all validation, it is executed to retrieve the result set R. In a final verification stage, the framework checks whether R logically entails the user's original query Q, the reference-free validation that distinguishes this pillar's notion of *logical integrity* from mere syntactic correctness.

3.5.1 Entailment scoring

An entailment score $E(Q, R)$ is computed by an LLM-as-a-judge, assessing both **schema consistency** (E_{schema}) and **data consistency** (E_{data}), as in Eq. 3.4.

$$E(Q, R) = \beta \cdot E_{\text{schema}} + (1 - \beta) \cdot E_{\text{data}} \quad \text{where} \quad \beta \in [0, 1] \tag{3.4}$$

3.5.2 Entailment decision

If $E(Q, R) \geq \theta_e$, the result is returned to the user and the KB is updated with the successful query–result pair. Otherwise, the entire pipeline is re-triggered with feedback on the failure reason (e.g., missing columns, incorrect filters). This final loop ensures the output truly matches the user's intent.

3.6 Mitigating Bias in LLM-based Evaluation

Using an LLM as a judge can introduce biases [23]. To mitigate this, the framework employs a structured, few-shot prompting strategy for all scoring tasks: prompts provide detailed instructions, scoring rubrics, and few-shot examples that anchor the LLM's judgment to task-aligned criteria, increasing the reliability and reproducibility of the evaluation while reducing the influence of the model's inherent biases.

3.7 Model Parameter Optimization

A systematic grid search determined the optimal thresholds ($\theta_r, \theta_s, \theta_e$) and weights ($\alpha, \beta, w_c, w_g, w_j, w_q$). Each parameter was varied and the combination yielding the highest validation accuracy was selected; the final values are given in Table 3.2.

Parameter	Value
θ_r	4
θ_s	5
θ_e	3
α	0.6
β	0.2
w_c	0.1
w_g	0.3
w_j	0.4
w_q	0.2

Table 3.2 Optimized parameters using grid search.

3.8 Evaluation

3.8.1 Experimental setup

We evaluate the framework on the **BIRD benchmark** [8]. Because large-scale, proprietary enterprise datasets are generally inaccessible for academic experiments, BIRD serves as an excellent proxy: it tests Text-to-SQL systems over diverse relational databases that mirror real-world complexity.

3.8.2 Results

We evaluate Claude, OpenAI, and DeepSeek API-based models, as well as an open-source DeepSeek model. Table 3.3 reports execution accuracy on BIRD. Across every model, enabling the reflection framework produces substantial gains over the corresponding non-reflective baseline (a 6–8% overall improvement), confirming that iterative reasoning and refinement are central to performance.

Model	Simple	Moderate	Challenging	Overall
Claude Models				
Claude-sonnet-4.5 (with reflection)	**88.97**	**50.65**	**32.41**	**72.03**
`Claude-sonnet-4.5 (w/o reflection)`	81.18	45.25	24.13	64.92
OpenAI API Models				
o1-preview (with reflection)	**75.89**	**48.49**	**18.62**	**62.19**
`o1-preview (w/o reflection)`	69.35	40.54	15.86	55.66
DeepSeek API Models				
deepseek-reasoner (with reflection)	**73.62**	**46.55**	**17.24**	**60.10**
`deepseek-reasoner` (w/o reflection)	62.90	40.54	12.41	51.88
Open Source Models (Local setup)				
DeepSeek-Coder-V2-Lite... (with reflection)	**43.55**	**21.62**	**3.44**	**35.02**
`DeepSeek-Coder-V2-Lite...` (w/o reflection)	30.65	13.51	0.0	22.64

Table 3.3 Accuracy (%) of various models on the BIRD dataset. Top performers in each category are highlighted.

3.8.3 Human evaluation

To complement the automatic evaluation, we conducted a qualitative human assessment of the correctness and semantic alignment of generated queries, focusing on the impact of the entailment check on subtle logical errors that syntax or basic semantic validation may miss. Table 3.4 shows a representative case in which entailment feedback corrects a division-by-zero and casting error that would otherwise return null results.

Stage	Analysis / SQL Snippet
User Query	List the lowest three eligible free rates for students in continuation schools.
Initial SQL	`SELECT ... (f.Count / f.Enrollment) ... WHERE s.SOCType LIKE '%Continuation%'`
Entailment Check Result	**Score**: 2 / 5 **Reason:** The result returns None values due to division by zero or nulls. Filters for positive enrollment and float casting are needed.
Refined SQL	`SELECT ... (CAST(f.Count AS REAL) / f.Enrollment) ... WHERE f.Enrollment > 0 AND s.SOCType LIKE '%Continuation%'`

Table 3.4 Example of SQL refinement driven by entailment feedback.

3.8.4 Comparison with state of the art

We benchmark against leading methods on the BIRD development set (Table 3.5). Reflect-SQL achieves an execution accuracy of 72.03%, a marked improvement over DIN-SQL [5] (50.72%), DAIL-SQL [20] (57.41%), and MAC-SQL [6] (59.59%), and is comparable to the recent CHASE-SQL architecture [7], establishing Reflect-SQL as a top-tier solution for complex database querying.

Method	Overall Accuracy
BIRD paper [8]	46.35
DIN-SQL [5]	50.72
DAIL-SQL [20]	57.41
MAC-SQL [6]	59.59
CHASE-SQL + Claude 3.5 Sonnet [7]	69.53
CHASE-SQL + Gemini 1.5 Pro [7]	**73.01**
Reflect-SQL + Claude 4.5 Sonnet (Ours)	**72.03**

Table 3.5 Comparison with state-of-the-art methods on the BIRD dev set (execution accuracy %).

3.9 Ablation Study

To quantify each component's contribution, we performed an ablation using `deepseek-chat` as the base model (Table 3.6). Removing any component degrades performance, and the self-reflection loop in SQL generation proves most critical: its removal causes the sharpest drop. These results affirm the effectiveness of the multi-stage, reflective design.

Variant	Simple	Moderate	Challenging	Overall
Full Model (Self-Reflect)	68.64	43.10	14.48	55.80
w/o Self-Reflection in SQL Generation	62.91	37.93	12.41	50.58
w/o Iterative Retrieval	67.02	40.30	10.34	53.58
w/o Entailment Check	64.10	41.59	13.10	52.47

Table 3.6 Ablation results showing the contribution of each module.

3.10 Summary

Reflect-SQL demonstrates that self-reflection, adapted to the structured domain of databases, is an effective route to reliable schema understanding. Its dynamic knowledge base disambiguates obscure schemas and adapts over time; its feedback-driven retrieval aligns vague queries with the schema; and its layered generation and entailment loops enforce logical integrity without gold-standard labels. The framework's effectiveness depends on the initial quality of the knowledge base and can face scalability challenges with extremely large schemas, and the automatic metrics for semantic correctness and entailment remain an evolving area of research. These observations, together with the data-understanding challenges of Chapter 2, motivate the research direction discussed in the concluding chapter.

Chapter 4

Future Directions

The preceding chapters traced tabular data understanding across two pillars (schema understanding and data understanding) and presented our contribution, Reflect-SQL [1], to the former. This chapter sets out the research agenda that follows from that work. It is deliberately extensive and organized in two parts. The first part consolidates what the schema-understanding pillar has achieved and identifies the problems that remain open, since these directly extend our completed and ongoing work. The second part charts the direction we intend to pursue next: the data-understanding pillar, where the survey's open challenges (Section 2.3) define the frontier and where a recurring theme (the architecture of evidence retrieval) suggests a concrete point of entry.

4.1 Future Directions in Schema Understanding

The schema-understanding pillar has matured along the three axes identified in Section 2.2. Obscure schemas are now routinely enriched with a semantic layer through knowledge-base and metadata construction; relevant schema elements are located through join-aware, iterative retrieval; and faithful queries are produced by multi-step pipelines that decompose, verify, and correct rather than generate in a single pass. Reflect-SQL advances this frontier by threading self-reflection through all three stages (knowledge-base construction, retrieval, and synthesis) and grounding it in the checkable signals that the SQL setting affords, reaching 72.03% execution accuracy on BIRD. Yet several problems remain unsolved, and they define the near-term agenda.

4.1.1 From static to continually evolving knowledge bases

A recurring limitation across the knowledge-base literature, and a constraint our own work shares, is that the semantic layer is treated as a *static* artifact: it is constructed once and thereafter fixed. Real schemas, however, drift: columns are added, business meanings shift, and each new query may reveal an undocumented facet of the data. The open problem is a *continually evolving* knowledge base that updates itself from query logs, execution feedback, and user corrections, versioning schema semantics as they change rather than assuming a one-time build. Such a knowledge base would turn schema

understanding from a preprocessing step into a lifelong-learning loop, in which every resolved query improves the system's model of the schema for the next one.

4.1.2 Retrieval at industry scale

Schema linking is accurate on academic benchmarks but becomes the binding constraint at industry scale, where a database may hold thousands of tables and tens of thousands of columns that cannot fit in a model's context window. The iterative, reflection-driven retrieval that Reflect-SQL employs is effective precisely because it reformulates the query when the retrieved context is judged insufficient, but every reformulation is another LLM call, so retrieval cost grows with the difficulty of the query. This mirrors a tension observed more broadly in structured retrieval, where coupling selection decisions to LLM invocations makes precision expensive [39, 40]. A promising direction is to reframe schema retrieval from a query-time *search* into addressing against *pre-computed* structure: materializing foreign-key graphs, column-value indices, and description embeddings offline so that schema linking navigates directly to the relevant elements at near-constant cost, reserving LLM calls for genuine disambiguation rather than routine traversal. Decoupling retrieval from repeated model invocation is, in our view, the key to schema understanding that scales.

4.1.3 Cross-database and multi-table join reasoning

Reflect-SQL, like most current systems, is evaluated over a single database at a time. Enterprise text-to-SQL workflows are considerably harder: they span multiple databases and dialects, require long foreign-key chains and deeply nested queries, and demand reasoning about which of many near-duplicate tables is authoritative [34]. Extending self-reflective synthesis to this setting, where the space of join paths is combinatorial and a single wrong join silently corrupts the result, is an open problem. Join-aware retrieval that preserves relationship structure (Section 2.2) is a necessary but not sufficient ingredient; the harder question is how reflection should reason about join *correctness* when execution alone cannot distinguish a plausible-but-wrong join from the intended one.

4.1.4 Reliable, reference-free verification

Reflect-SQL's central bet is that reflection can be grounded in checkable signals (syntax, execution, and result–intent entailment scored by an LLM judge) so that queries can be validated without a gold reference. The reliability of that judgment is itself an open problem. LLM-as-a-judge is known to carry position, verbosity, and self-preference biases [23], and a judge that is confidently wrong is more dangerous than one that abstains. Future work should pursue calibrated, reference-free verification: quantifying when a self-assessment can be trusted, learning when to escalate to a human or to additional evidence rather than accepting a flawed query, and stress-testing the semantic-validation dimensions against adversarial and near-miss queries where syntactic validity and semantic faithfulness diverge.

4.1.5 The cost of reflection

Reflection buys accuracy at the price of latency: iterative retrieval and iterative synthesis, each followed by multi-dimensional validation, multiply the number of LLM calls per query. For interactive analytics this cost is a deployment barrier. The open question is how to preserve the accuracy gains of reflection while collapsing its cost, for instance by distilling multi-step reflection into fewer passes, caching validated sub-queries and retrieval decisions across a session, and learning a stopping policy that halts refinement once marginal gains vanish. Efficiency here is not a secondary concern but a first-class research problem, and it foreshadows the efficiency–accuracy tension that dominates the data-understanding pillar below.

4.1.6 Beyond SQL

Finally, schema understanding is broader than SQL generation. Real analytical requests are frequently abstract and goal-oriented ("why did revenue dip last quarter?") rather than a clean specification of a query, and answering them may call for dataframe or general code generation rather than a single SQL statement [34, 35]. Inferring the underlying analytical intent, and mapping it onto whichever execution substrate fits, generalizes the schema-linking problem and forms a natural bridge to the data-understanding pillar, to which we now turn.

4.2 Setting a Direction for Data Understanding

Having established schema understanding through Reflect-SQL, our next research phase turns to the second pillar: data understanding, and concretely Table Question Answering, the problem of reasoning faithfully over the *content* of tables. The survey literature frames this frontier through three challenges (Section 2.3): limited scope beyond retrieval and arithmetic, a lack of robustness on complex inputs, and limited generalization across representations. These challenges tell us *what* is missing; the purpose of this section is to identify *where* to begin. Reviewing how the field currently reasons over table content surfaces a recurring bottleneck (the architecture by which evidence is delivered to the model) that offers a concrete and, we argue, high-leverage point of entry.

4.2.1 Evidence retrieval as the central bottleneck

Before a model can reason over a table, the relevant cells must be placed in front of it. As real-world tables grow to hundreds of rows and columns, this delivery step, not the model's reasoning ability, becomes the binding constraint: an LLM handed a large table must locate a handful of relevant cells amid overwhelmingly irrelevant content, and both context budget and reasoning accuracy degrade sharply as noise grows, the "lost-in-the-middle" effect [41]. Effective table reasoning therefore hinges on retrieval: isolating the minimal cell evidence a model needs before asking it to reason.

The difficulty is that prevailing approaches couple cell selection to LLM invocation. Full-table linearization performs no selection at all, forcing the model to filter in-context and exhausting the budget on large inputs. Decomposition pipelines such as Chain-of-Table [40] and DATER [39] select rows and columns through a sequence of LLM calls, so every filtering decision costs an additional invocation and errors compound across stages. Graph-based agentic navigators such as GraphOTTER [38] traverse a structured representation of the table but score each traversal hop with the LLM, tying latency to table size with no way to skip irrelevant regions without first visiting them. In each case, improving retrieval precision demands more LLM calls while reducing calls risks discarding evidence, so an efficiency–accuracy tension appears fundamental.

We see this tension as an artifact of design rather than an inherent constraint, and this observation frames the direction we intend to pursue. A table is not only a bag of cells to be searched but a *coordinate system*: most answers are locatable at the intersection of particular rows and columns, and the groupings, statistics, and semantic partitions needed to navigate to that intersection do not depend on the query and can be pre-computed offline. This suggests reframing evidence retrieval from a query-time search into addressing against a pre-materialized index (structural, statistical, and semantic) so that cell selection is resolved cheaply and the LLM is invoked primarily for final synthesis. Such a decoupling of selection from reasoning is the direct content-side analogue of the schema-retrieval direction of Section 4.1, and unifying the two under a single "retrieval-as-addressing" principle is the through-line we intend to develop across both pillars.

4.2.2 Robustness on complex table structures

The efficiency bottleneck is most acute precisely where the survey reports the largest human–model gaps: hierarchical, long, and multi-table inputs, where accuracies fall below 50% against human performance around 83% on benchmarks such as HiTab [9] and MULTIHIERTT [10]. Complex structure compounds the retrieval problem: multi-row headers carry span labels that flat serialization loses, and merged regions create implicit repetition that corrupts naive cell addressing. A robust system must therefore condition the table before reasoning (propagating hierarchical headers and normalizing merged cells) and carry that structure through to the representation, whether via specialized encoders in the tradition of TAPAS [42] and TaBERT [43] or via structure-aware indices built over serialized views. Robustness to input complexity, rather than incremental gains on already-saturated simple benchmarks, is where we intend to concentrate.

4.2.3 Reasoning beyond retrieval and arithmetic

Once evidence can be delivered reliably, the harder question is what the model does with it. Many benchmarks are constructed by translating SQL or arithmetic expressions into natural language, so they reward matching synonyms to headers rather than genuine reasoning, and they are approaching saturation. The frontier is diagnostic, predictive, and prescriptive analysis (identifying insights, forecasting, and responding to abstract, goal-oriented requests) which requires understanding the table's overall

context and the user's intent rather than surface retrieval [34, 35]. This is also where the two pillars converge: inferring analytical intent (a schema-understanding capability) and reasoning over content (a data-understanding capability) are two halves of the same problem, and a system that first understands structure and then reasons over content is the long-term target.

4.2.4 Generalization across representations

Finally, model performance varies by several percentage points with the input format, and systems transfer poorly across the diverse representations used by different benchmarks (Section 2.1) [3]. Promising remedies include standardizing serialization options, deriving serialized views from images of complex tables, and training on serialization-to-serialization tasks that convert one format into another to build robustness. Any evidence-retrieval architecture we develop should be evaluated for representation-invariance from the outset, so that gains do not prove brittle to a change of input format.

4.3 Summary

Taken together, these directions describe a coherent trajectory. In schema understanding, the immediate work is to make the semantic layer continually evolving, to scale retrieval by addressing rather than search, to extend faithful synthesis to cross-database joins, and to make reflection both reliable and cheap. In data understanding, our next pillar, the point of entry is the evidence-retrieval bottleneck, approached through the same decoupling of selection from reasoning, with robustness on complex structures and higher-order reasoning as the targets and representation-invariance as a standing requirement. The unifying thread is that reliable tabular reasoning depends less on ever-larger models than on the architecture that delivers the right structure and the right evidence to them.

Chapter 5

Conclusion

This report has examined *tabular data understanding* with large language models through two complementary pillars (schema understanding and data understanding) and situated our own contribution within them.

Chapter 2 established the common ground on which both pillars rest: because language models consume one-dimensional token sequences while tables are inherently two-dimensional, every pipeline must first commit to a representation, and this choice (serialization, data schema, image, or a specialized table encoder) conditions everything downstream. Building on that foundation, the survey organized the field into schema understanding, the mapping of natural-language intent onto a structural blueprint, and data understanding, the problem of reasoning faithfully over table content. Across both, a consistent picture emerged: single-pass methods have given way to multi-step pipelines that decompose, retrieve, verify, and correct, and the binding constraints are increasingly architectural (how structure and evidence are delivered to the model) rather than a matter of raw model capacity.

Chapter 3 presented our contribution to the schema-understanding pillar, Reflect-SQL [1], which threads self-reflection through all three stages of a Text-to-SQL pipeline: it constructs a semantic knowledge base over the schema, retrieves relevant elements iteratively, and synthesizes queries under syntactic and multi-dimensional semantic validation, using an LLM-as-a-judge to score correctness without a gold reference. Grounding reflection in the checkable signals that the SQL setting affords (execution, syntax, and result–intent entailment) allows the system to reach 72.03% execution accuracy on the industry-scale BIRD benchmark, and demonstrates that reference-free, self-critical generation is a viable path for structured tasks.

Chapter 4 then set out the research agenda that follows. On the schema side, the priorities are to make the knowledge base continually evolving rather than static, to scale retrieval by addressing pre-computed structure rather than searching at query time, to extend faithful synthesis to cross-database joins, and to make reflection both reliable and inexpensive. On the data-understanding side, our next pillar, the target task is Table Question Answering and the point of entry is the evidence-retrieval bottleneck, approached through the same principle of decoupling selection from reasoning, with robustness on complex table structures and higher-order reasoning as the targets.

The unifying thesis of this report is that progress in tabular data understanding depends less on ever-larger models than on the architecture that supplies them with the right structure and the right evidence. Reflect-SQL is a concrete step in that direction for schema understanding; carrying the same principle into data understanding is the trajectory we intend to pursue.

Bibliography

[1] A. Jain and M. Shrivastava, "Reflect-SQL: A self-reflection based framework for text-to-sql," in *Trends and Applications in Knowledge Discovery and Data Mining (PAKDD)*, ser. Lecture Notes in Computer Science. Singapore: Springer Nature Singapore, 2027, pp. 74–86.

[2] X. Wu, A. Ritter, and W. Xu, "Tabular data understanding with LLMs: A survey of recent advances and challenges," *arXiv preprint arXiv:2508.00217*, 2025.

[3] Y. Sui, M. Zhou, M. Zhou, S. Han, and D. Zhang, "Table meets LLM: Can large language models understand structured table data? a benchmark and empirical study," in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*, 2024.

[4] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," *arXiv preprint arXiv:1709.00103*, 2017.

[5] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai *et al.*, "MAC-SQL: A multi-agent collaborative framework for text-to-SQL," in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024.

[7] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei *et al.*, "CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL," *arXiv preprint arXiv:2410.01943*, 2024.

[8] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li *et al.*, "Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs (BIRD)," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[9] Z. Cheng, H. Dong, Z. Wang, R. Jia, J. Guo, Y. Gao, S. Han, J.-G. Lou, and D. Zhang, "HiTab: A hierarchical table dataset for question answering and natural language generation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[10] Y. Zhao, Y. Li, C. Li, and R. Zhang, "MULTIHIERTT: Numerical reasoning over multi hierarchical tabular and textual data," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[11] A. Su, A. Wang, C. Ye, C. Zhou, G. Zhang *et al.*, "TableGPT2: A large multimodal model with tabular data integration," *arXiv preprint arXiv:2411.02059*, 2024.

[12] E. Lobo *et al.*, "Matching business glossaries to databases: A semantic and few-shot learning approach," in *Proceedings of the 2023 International Conference on Management of Data (SIGMOD)*, 2023.

[13] Y. Gao *et al.*, "Automatic metadata generation for text-to-SQL," *Proceedings of the VLDB Endowment*, 2025.

[14] Z. Ma *et al.*, "Enhancing text-to-SQL with domain knowledge injection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[15] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[17] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv:2310.11511*, 2023.

[18] X. Xu, C. Liu, and D. Song, "SQLNet: Generating structured queries from natural language without reinforcement learning," *arXiv preprint arXiv:1711.04436*, 2017.

[19] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[20] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-SQL empowered by large language models: A benchmark evaluation (DAIL-SQL)," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, 2024.

[21] R. Sun, S. O. Arik, A. Muzio, L. Miculicich, S. Gundabathula *et al.*, "SQLPrompt: In-context text-to-SQL with minimal labeled data," in *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

[22] Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating LLM hallucination via self-reflection," in *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

[23] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[24] P. Pasupat and P. Liang, "Compositional semantic parsing on semi-structured tables," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2015.

[25] M. Iyyer, W.-t. Yih, and M.-W. Chang, "Search-based neural structured learning for sequential question answering," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

[26] F. Zhu, W. Lei, Y. Huang, C. Wang, S. Zhang *et al.*, "TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[27] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Borova *et al.*, "FinQA: A dataset of numerical reasoning over financial data," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[28] W. Chen, H. Zha, Z. Chen, W. Xiong, H. Wang, and W. Y. Wang, "HybridQA: A dataset of multi-hop question answering over tabular and textual data," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020.

[29] V. Pal, A. Yates, E. Kanoulas, and M. de Rijke, "MultiTabQA: Generating tabular answers for multi-table question answering," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[30] L. Nan, C. Hsieh, Z. Mao, X. V. Lin, N. Verma *et al.*, "FeTaQA: Free-form table question answering," in *Transactions of the Association for Computational Linguistics*, 2022.

[31] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang *et al.*, "TabFact: A large-scale dataset for table-based fact verification," in *International Conference on Learning Representations (ICLR)*, 2020.

[32] A. Parikh, X. Wang, S. Gehrmann, M. Faruqui, B. Dhingra *et al.*, "ToTTo: A controlled table-to-text generation dataset," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[33] Y. Zhao, Z. Qi, L. Nan, B. Mi, Y. Liu *et al.*, "QTSUMM: Query-focused summarization over tabular data," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[34] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows," in *arXiv preprint arXiv:2411.07763*, 2024.

[35] X. He, M. Zhou, X. Xu, X. Ma, R. Ding *et al.*, "Text2Analysis: A benchmark of table question answering with advanced data analysis and unclear queries," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[36] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[37] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[38] Q. Li, C. Huang, S. Li, Y. Xiang, D. Xiong, and W. Lei, "GraphOTTER: Evolving LLM-based graph reasoning for complex table question answering," in *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.

[39] Y. Ye, B. Hui, M. Yang, B. Li, F. Huang, and Y. Li, "Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning," in *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.

[40] Z. Wang, H. Zhang, C.-L. Li, J. M. Eisenschlos, V. Perot, Z. Wang, L. Miculicich, and T. Pfister, "Chain-of-table: Evolving tables in the reasoning chain for table understanding," in *International Conference on Learning Representations (ICLR)*, 2024.

[41] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[42] J. Herzig, P. K. Nowak, T. Müller, F. Piccinno, and J. Eisenschlos, "TAPAS: Weakly supervised table parsing via pre-training," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[43] P. Yin, G. Neubig, W.-t. Yih, and S. Riedel, "TaBERT: Pretraining for joint understanding of textual and tabular data," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.